

zTSkut: A web application for predicting thermoelectric performance in skutterudites with deep learning

Victor Posligua^{1,2}, Karina Landivar^{1,3}, Elena R. Remesal¹, Gerda Rogl⁴, Peter F. Rogl⁴, Javier Fdez Sanz¹, Jesus Prado-Gonjal⁵, Antonio M. Márquez¹, and Jose J. Plata¹

¹ Departamento de Química Física, Facultad de Química, Universidad de Sevilla, Seville, Spain ² Institute for Functional Intelligent Materials, National University of Singapore, 4 Science Drive 2, 117544, Singapore ³ Dipartimento di Fisica "Aldo Pontremoli", Università degli Studi di Milano, Via Celoria, 16, I-20133 Milano, Italy ⁴ Institute of Materials Chemistry, University of Vienna, Waehringerstraße 42, Wien, Austria ⁵ Departamento de Química Inorgánica, Universidad Complutense de Madrid, Madrid, Spain ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 14 July 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

zTSkut is a Python-based web application for predicting the thermoelectric figure of merit, zT, of skutterudite-based compositions. The software provides both a browser interface for single-composition prediction and a batch-screening workflow based on CSV, JSON or Excel .xlsx input files. Users specify the anion, cation and filler species, their corresponding fractions, carrier concentration at 300 K, temperature and carrier type. zTSkut then generates the same compositional descriptors used by the trained neural-network model, applies the saved feature scaling from the original training workflow and returns predicted zT values. For batch predictions, users can download the input table augmented with a predicted_zT column in CSV, JSON or Excel .xlsx format.

The backend generates the compositional descriptors required by the trained neural-network model, orders them according to the original feature list, applies the saved StandardScaler from the training workflow, and returns predicted zT values. This ensures that user predictions follow the same preprocessing pathway as the published model-development study. zTSkut is built with a FastAPI backend and a lightweight HTML/CSS/JavaScript frontend, and includes input validation to catch common formatting errors.

Statement of need

Skutterudites are an important family of thermoelectric materials because their structure allows chemical tuning through filler, cation and anion substitution. This large compositional flexibility makes them suitable for computational screening, but it also creates a practical barrier for experimental researchers: evaluating many candidate compositions using first-principles transport calculations or synthesis is expensive and slow. Machine learning (ML) models can help prioritise promising candidates, but they are only useful to the broader community if they can be accessed without requiring users to reproduce the full training and descriptor-generation workflow.

zTSkut addresses this need by exposing the trained skutterudite zT model from Posligua et al. (Posligua et al., 2026) through a simple web interface. The purpose of zTSkut is not to replace that model-development study, but to make the trained predictor usable for

41 screening new skutterudite candidates. The app allows users to test individual compositions
42 interactively or upload large candidate lists for batch prediction. This is particularly useful
43 for researchers designing filled or multi-filled skutterudites and looking for a rapid first-pass
44 estimate of thermoelectric performance before more expensive experimental or computational
45 validation.

46 State of the field

47 Several ML models have been developed for thermoelectric materials, including models trained
48 on broad thermoelectric datasets or on specific material families. However, these models
49 are often published primarily as methodological studies, with limited user-facing software for
50 applying the trained model to new candidate compositions. In practice, reusing such models
51 can require reproducing the original descriptor generation, feature ordering, scaling and loading
52 workflow, which creates a barrier for experimental or computational users who simply want to
53 screen new compositions.

54 This follows a broader trend in scientific computing, where AI models are increasingly used as
55 surrogates to accelerate exploration of large parameter spaces, provided that predictions remain
56 within the model domain and promising outcomes are verified with higher-fidelity calculations
57 or experiments (Dongarra et al., 2026).

58 General purpose ML platforms and Python libraries such as TensorFlow/Keras (Abadi et al.,
59 2015; Chollet & others, 2015) and scikit-learn (Pedregosa et al., 2011) provide the underlying
60 infrastructure for training and deploying models, but they do not provide a domain-specific
61 interface for skutterudite zT prediction. Similarly, web frameworks such as FastAPI (Ramírez,
62 2018) make deployment possible, but they do not encode the thermoelectric descriptor logic,
63 saved model artefacts or input conventions required for this specific skutterudite predictor.

64 zTskut therefore fills a focused gap: it packages the trained skutterudite model (Posligua
65 et al., 2026) into a reproducible and user-facing prediction tool. Rather than contributing
66 to a general platform, zTskut was built as a lightweight dedicated application because the
67 model requires a specific descriptor representation, fixed feature order and saved training scaler
68 to ensure consistency with the peer-reviewed workflow. This narrow scope is intentional: it
69 keeps the tool easy to use, easy to deploy and directly aligned with the skutterudite screening
70 problem.

71 Software design

72 The main design goal of zTskut is to make the trained skutterudite zT model accessible while
73 preserving the exact preprocessing workflow used during model development. For this reason,
74 the software separates the user interface from the prediction pipeline but keeps the deployment
75 structure simple enough to run locally or on a lightweight web service.

76 The backend is implemented with FastAPI, with both the single-system form and the batch
77 uploads passing through the same prediction workflow. Batch inputs can be provided as CSV,
78 JSON or Excel .xlsx files; JSON and Excel inputs are converted internally to the same CSV-style
79 table before descriptor generation and prediction. The prediction script reads the input, validates
80 numerical fields, replaces empty optional fields with zero and generates the compositional
81 descriptors required by the trained model. Descriptor generation uses RDKit (Landrum & others,
82 2024) to parse element-level chemical inputs and obtain valence-electron descriptors, while
83 Mendeleev (Mentel, 2014--) and Pymatgen (Ong et al., 2013) provide elemental properties,
84 including atomic masses as well as ionisation-energy, electron-affinity and electronegativity
85 values. The resulting descriptor table is ordered according to feature_columns_ZT.txt,
86 transformed with scaler_ZT.joblib, which stores the feature-scaling parameters fitted during

87 model training, and finally evaluated with the trained `model_keras_skutt.h5` model. The
88 same end-to-end prediction workflow is summarised in **Figure 1**.

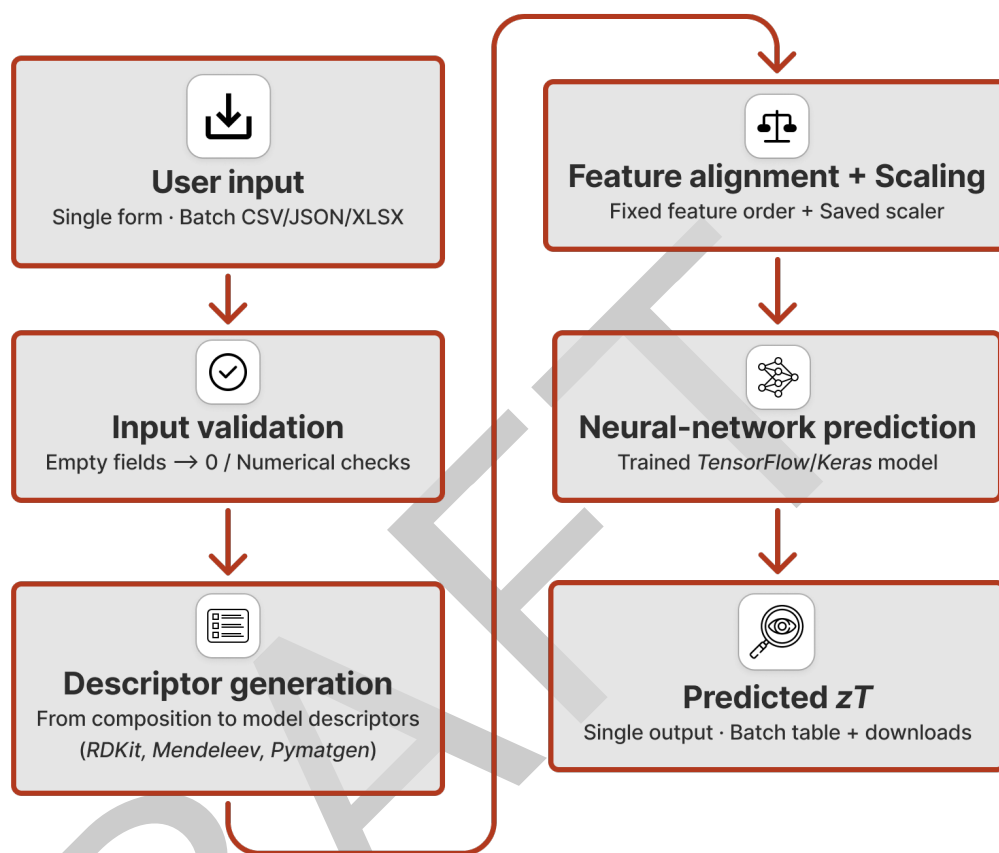


Figure 1: zTSkut prediction workflow. Single-composition inputs and batch CSV, JSON or Excel .xlsx inputs are processed through the same backend pipeline, including input validation, descriptor generation with RDKit, Mendelev and Pymatgen, feature alignment using `feature_columns_ZT.txt`, scaling with `scaler_ZT.joblib` and prediction with the trained TensorFlow/Keras model.

89 A deliberate trade-off was made to keep the software lightweight and transparent rather than
90 building a more complex package architecture. The current implementation uses a small
91 number of files so that it can be deployed easily on Render and run locally with a single
92 Uvicorn command. This is important for the intended audience: researchers who want to use
93 the model for candidate screening should not need to install a large framework or understand
94 the full training workflow. At the same time, the repository includes tests for basic web app
95 functionality, prediction through the backend endpoint and input validation, so that reviewers
96 and users can verify that the core workflow is functioning.

97 The frontend is implemented as a lightweight HTML/CSS/JavaScript interface without
98 additional frontend framework dependencies. This avoids extra deployment complexity and
99 keeps the interface portable.

100 Software functionality

101 The current implementation provides the following functionality:

- 102 ■ Single-system prediction through a browser form
- 103 ■ Batch prediction from CSV, JSON and Excel .xlsx files

- Downloadable batch prediction results in CSV, JSON, and Excel .xlsx formats
- Preservation of uploaded input fields in batch outputs, with predictions added as a new predicted_ZT column
- Automatic treatment of empty optional composition and fraction fields as zero
- Generation of the compositional descriptor set expected by the trained model
- Application of the saved training scaler and fixed feature order
- Input validation for numerical fields such as fractions, temperature, carrier concentration and carrier type
- Downloadable citation files for users of the model
- Local execution through FastAPI as well as deployment as a web app

For batch predictions, users can download the CSV template, add one candidate composition per row or record and upload the completed file as CSV, JSON or Excel .xlsx. The app returns one predicted zT value per system and provides downloadable result files in which the original input fields are preserved and the prediction is added as predicted_ZT.

Research impact statement

The scientific model served by zTskut has already been peer reviewed and published in Journal of Materials Chemistry A (Posligua et al., 2026). In that study, the model was trained on a curated skutterudite dataset and evaluated using internal validation, independent external testing, comparison with experimental trends and physical interpretation through SHAP analysis and first-principles calculations.

zTskut extends the impact of that work by converting the published model into an accessible screening tool. This is particularly useful for experimental and computational researchers considering new filled or multi-filled skutterudite compositions, because the software provides rapid first-pass predictions before synthesis or expensive transport calculations. The live web app, documented CSV/JSON/Excel input formats, downloadable result files, citation files, local execution workflow and test suite make the model easier to reuse beyond the original manuscript and support its integration into future thermoelectric screening workflows.

Availability and use

The web application is available at <https://ztskut.onrender.com/>. The repository provides the source code, trained model files, saved scaler, feature-column definition, CSV template, example inputs, citation files and tests for the web endpoint, supported upload formats, downloadable batch outputs and input validation. Users can access the model either through the deployed web interface or by running the same backend prediction workflow locally using the example CSV files.

zTskut is intended as a screening and prioritisation tool. Predictions should be interpreted within the chemical and thermoelectric domain represented by the model-development dataset. In particular, predictions for compositions far outside the training chemistry, unusual carrier concentrations or temperatures, systems affected by secondary phases, or very high-zT regimes should be treated with caution and validated experimentally or with higher-fidelity calculations.

AI usage disclosure

No AI tools were used to assist with the code and web app. All scientific content, software behaviour, tests and final text were reviewed, edited and validated by the authors. The trained model, dataset, scientific interpretation and research conclusions are based on the peer-reviewed work (Posligua et al., 2026).

Acknowledgements

- The trained model used by zTSkut was developed as part of the skutterudite thermoelectrics study reported in (Posligua et al., 2026). This work was funded by grant PID2022-138063OB-I00 funded by MICIU/AEI/10.13039/501100011033 and by FEDER, UE, and by grants TED2021-130874B-I00 and TED2021-129569A-I00 funded by MICIU/AEI/10.13039/501100011033 and by the European Union NextGenerationEU/PRTR.
- The authors acknowledge the computer resources at Lusitania (Cenits-COMPUTAEX), Red Española de Supercomputación, RES (QHS-2024-1-0022 and QHS-2024-2-0020), and Albaicín (Centro de Servicios de Informática y Redes de Comunicaciones – CSIRC, Universidad de Granada).
- Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., ... Zheng, X. (2015). *TensorFlow: Large-scale machine learning on heterogeneous systems*. <https://www.tensorflow.org/>
- Chollet, F., & others. (2015). *Keras*. <https://keras.io>
- Dongarra, J., Reed, D., & Gannon, D. (2026). Scientific computing in an AI world. *Science*, 392(6804). <https://doi.org/10.1126/science.aef4214>
- Landrum, G., & others. (2024). *RDKit: Open-source cheminformatics*. <https://doi.org/10.5281/zenodo.14535873>
- Mentel, Ł. (2014--). *Mendeleev – a python resource for properties of chemical elements, ions and isotopes*. <https://github.com/lmmentel/mendeleev>
- Ong, S. P., Richards, W. D., Jain, A., Hautier, G., Kocher, M., Cholia, S., Gunter, D., Chevrier, V. L., Persson, K. A., & Ceder, G. (2013). Python materials genomics (pymatgen): A robust, open-source python library for materials analysis. *Computational Materials Science*, 68, 314–319. <https://doi.org/10.1016/j.commatsci.2012.10.028>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Posligua, V., Landivar, K., Remesal, E. R., Rogl, G., Rogl, P. F., Fdez. Sanz, J., Prado-Gonjal, J., Márquez, A. M., & Plata, J. J. (2026). Deep learning framework for accurate prediction and high-throughput search of the thermoelectric figure of merit in skutterudites. *Journal of Materials Chemistry A*. <https://doi.org/10.1039/D5TA08841K>
- Ramírez, S. (2018). *FastAPI*. <https://fastapi.tiangolo.com/>